

MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Balan Gabriel FAF-241

Report

Laboratory Work No. 5

Operating Systems

Topic: The File System with Your Own Hands

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

1 Purpose of the Work 3

2 Part 1 – The Tree and the File Types 3

2.1 Observations 3

3 Part 2 – Inodes and Links 5

3.1 Observations 5

4 Part 3 – Permissions 7

4.1 Observations 8

5 Part 4 – Disks and the Kernel as Files 9

5.1 Observations 9

6 Conclusion 11

1. Purpose of the Work

The purpose of this laboratory work is to inspect the Linux file system directly and connect the theory with what the operating system actually shows. I worked with the directory tree, file types, inodes, hard and symbolic links, permissions, disks, and the information exposed by `/proc`. The main idea was to see how all of these concepts behave in practice instead of only reading about them.

2. Part 1 – The Tree and the File Types

I started as my normal user and created a separate directory for the lab. Then I inspected some common system paths, device files, and the sizes of important directories.

Commands

```
whoami
mkdir -p ~/os-lab5 && cd ~/os-lab5
ls /
ls -ld /etc /etc/passwd /bin /dev/null /tmp
ls -l /dev | head
file /usr/bin/ls /etc/passwd /bin
du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
```

2.1. Observations

1. For each path in the second line, write its type from the first letter of `ls -l`.

The results were:

- `/etc` starts with `d`, so it is a directory.
- `/etc/passwd` starts with `-`, so it is a regular file.
- `/bin` starts with `l`, so it is a symbolic link.
- `/dev/null` starts with `c`, so it is a character device.
- `/tmp` starts with `d`, so it is a directory.

2. What is `/bin` on your system: a directory or a link, and to where?

On my system, `/bin` is not a real directory by itself. It is a symbolic link to `/usr/bin`. This is visible directly in the output as `/bin -> usr/bin`.

3. Find one character device and one block device in `/dev`.

A character device is `/dev/null`. A block device on my system is `/dev/sda` (and its partition `/dev/sda2`). Character devices are accessed as streams of bytes, while block devices represent storage that is handled in blocks.

4. Which is bigger on your laptop, `/var/log` or `/usr`, and by roughly how much?

```
findredhundred@Ubuntu:~$ whoami
findredhundred
findredhundred@Ubuntu:~$ mkdir -p ~/os-lab5 && cd ~/os-lab5
findredhundred@Ubuntu:~/os-lab5$ ls /
bin   cdrom  etc   lib    lost+found  mnt  proc  run   snap  sys  usr
boot  dev    home  lib64  media       opt  root  sbin  srv   tmp  var
findredhundred@Ubuntu:~/os-lab5$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx   1 root root     7 Apr 20 08:46 /bin -> usr/bin
crw-rw-rw-   1 root root  1, 3 Oct  5 14:27 /dev/null
drwxr-xr-x 146 root root 12288 Oct  2 13:02 /etc
-rw-r--r--   1 root root  2796 Sep 28 23:24 /etc/passwd
drwxrwxrwt  18 root root   660 Oct 12 19:05 /tmp
findredhundred@Ubuntu:~/os-lab5$ ls -l /dev | head
total 0
crw-r--r--  1 root           root     10, 235 Oct  5 14:27 autofs
drwxr-xr-x  2 root           root        480 Oct 12 17:21 block
drwxr-xr-x  2 root           root         80 Oct  5 14:27 bsg
crw-------  1 root           root     10, 234 Oct  5 14:27 btrfs-control
drwxr-xr-x  3 root           root         60 Oct  5 14:27 bus
lrwxrwxrwx+ 1 root           root          3 Oct  5 14:27 cdrom -> sr0
drwxr-xr-x  2 root           root       3780 Oct 12 16:52 char
crw-------  1 root           tty       5,   1 Oct  5 14:27 console
lrwxrwxrwx  1 root           root         11 Oct  5 14:27 core -> /proc/kcore
findredhundred@Ubuntu:~/os-lab5$ file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: symbolic link to ../lib/cargo/bin/coreutils/ls
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin
findredhundred@Ubuntu:~/os-lab5$ du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
75M     /var/log
6.1G    /usr
findredhundred@Ubuntu:~/os-lab5$
```

Figure 1.1 – Inspecting the Linux directory tree, file types, and directory sizes.

`/usr` is much bigger. In my output, `/var/log` was about 75 MB while `/usr` was about 6.1 GB. So `/usr` is roughly 6 GB larger, or around 80 times bigger.

3. Part 2 – Inodes and Links

Next I worked with inodes and links. I created a normal file, then a hard link and a symbolic link, and removed the original names to see what actually disappeared.

Commands

```
echo hello > notes.txt
stat notes.txt
ls -li notes.txt
ln notes.txt hard.txt
ls -li notes.txt hard.txt
rm notes.txt
cat hard.txt ; ls -li hard.txt
ln -s hard.txt soft.txt
ls -li hard.txt soft.txt
cat soft.txt
rm hard.txt
cat soft.txt ; ls -l soft.txt
mkdir d ; ls -ld d
mkdir d/x d/y ; ls -ld d
echo x > one.txt ; ls -l one.txt ; du -h one.txt
truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
rm big.img
```

3.1. Observations

1. Which number in `stat` is the inode, and which tells how many names the file has?

In my output, the inode number of `notes.txt` was `410161`. The `Links` field tells how many hard-link names point to that inode. At first it was 1, and after creating `hard.txt` it became 2.

2. After `rm notes.txt`, why could you still read `hard.txt`? What does `rm` really remove?

I could still read `hard.txt` because both names were pointing to exactly the same inode and the same file data. Removing `notes.txt` only removed that directory entry, basically one name for the file. The actual data stays until the last hard link to the inode is removed.

3. Why did `cat soft.txt` fail at the end, although `soft.txt` still exists? How big is `soft.txt` and why exactly that size?

`soft.txt` was only a symbolic link that stored the target name `hard.txt`. After I deleted `hard.txt`, the link still existed, but its target did not, so it became a broken symbolic link

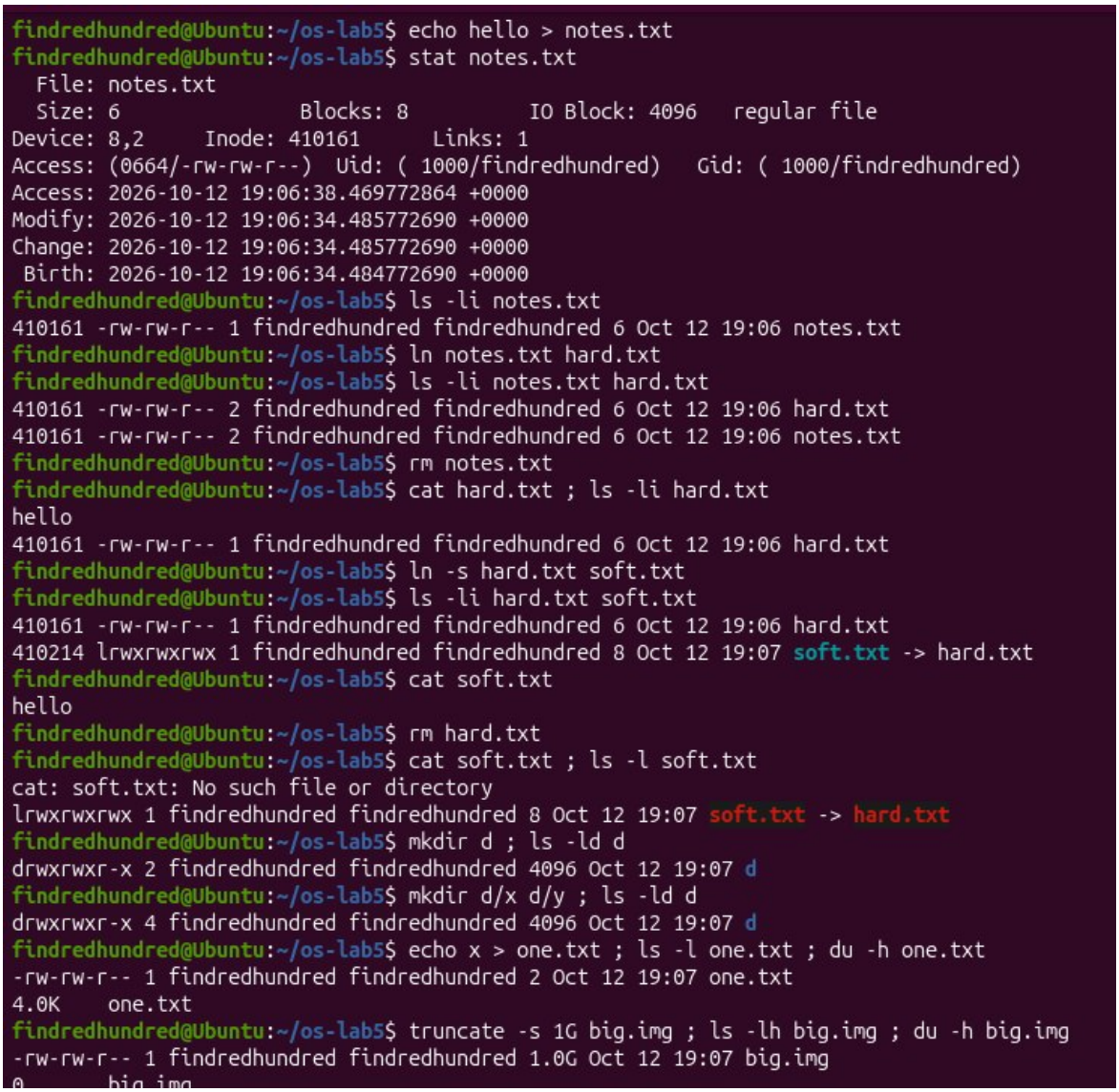


Figure 2.1 – Inodes, hard links, symbolic links, directory link counts, and sparse-file size.

and `cat` failed. The link was 8 bytes because the text `hard.txt` itself is 8 characters long.

4. A new directory has 2 links, and 4 after you create two subdirectories. Explain where each link comes from.

A new directory starts with 2 links: its own `.` entry, and the name for that directory in its parent. Each new subdirectory also contains a `..` entry pointing back to the parent. Since I created `x` and `y`, the parent directory gained two more links, so the count became 4.

5. Why does a 2-byte file use 4.0K, and a 1 GB file use 0?

The 2-byte file still occupies one filesystem block, so `du` reports 4.0K even though the logical file size is only 2 bytes. The 1 GB file created with `truncate` is sparse: its logical size is 1 GB, but no real disk blocks were allocated for the empty area yet, so `du` reported 0.

4. Part 3 – Permissions

In this part I changed file and directory permissions and then checked how Linux actually enforced them. I also looked at the setuid bit on `passwd`, the sticky bit on `/tmp`, and the permissions of `/etc/shadow`.

Commands

```
echo secret > s.txt
chmod 600 s.txt ; ls -l s.txt
chmod 000 s.txt ; cat s.txt
chmod u+r s.txt ; cat s.txt
umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
mkdir box ; echo hi > box/f
chmod u=r,go= box ; ls box ; cat box/f
chmod u=x,go= box ; ls box ; cat box/f
chmod 755 box
ls -l /usr/bin/passwd ; ls -ld /tmp
cat /etc/shadow
ls -l /etc/shadow
```

```
findredhundred@Ubuntu:~/os-lab5$ echo secret > s.txt
findredhundred@Ubuntu:~/os-lab5$ chmod 600 s.txt ; ls -l s.txt
-rw------- 1 findredhundred findredhundred 7 Oct 12 19:09 s.txt
findredhundred@Ubuntu:~/os-lab5$ chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied
findredhundred@Ubuntu:~/os-lab5$ chmod u+r s.txt ; cat s.txt
secret
findredhundred@Ubuntu:~/os-lab5$ umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
0002
-rw-rw-r-- 1 findredhundred findredhundred    0 Oct 12 19:09 new.txt
drwxrwxr-x 2 findredhundred findredhundred 4096 Oct 12 19:09 newdir
findredhundred@Ubuntu:~/os-lab5$ mkdir box ; echo hi > box/f
findredhundred@Ubuntu:~/os-lab5$ chmod u=r,go= box  ; ls box ; cat box/f
f
cat: box/f: Permission denied
```

Figure 3.1 – File permissions, `umask`, and the directory read/execute experiment.

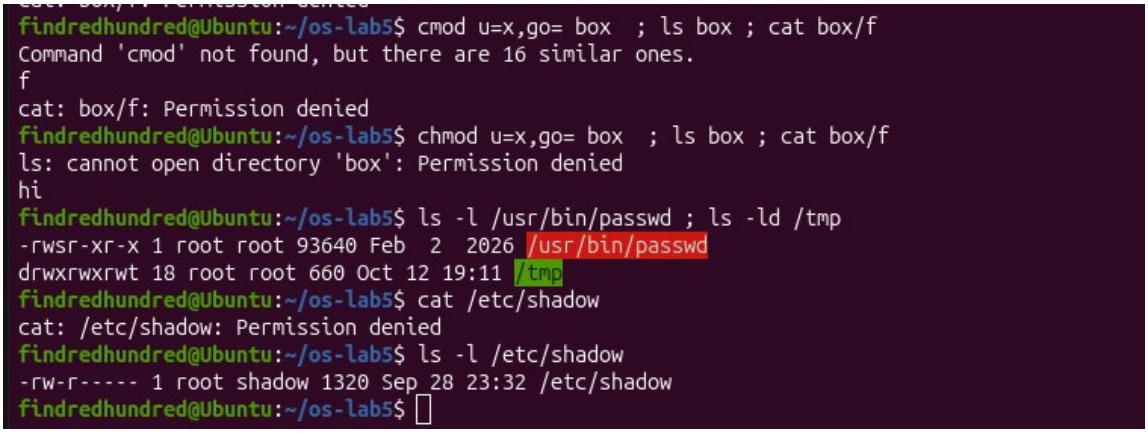


Figure 3.2 – Setuid on `/usr/bin/passwd`, the sticky bit on `/tmp`, and restricted access to `/etc/shadow`.

4.1. Observations

1. Write 600, 644, 755 and 700 as rwx strings, and give one real use for each.

- `600 = rw----`: useful for a private file that only the owner should read or modify, for example a private key or personal configuration file.
- `644 = rw-r-r-`: a normal text or configuration file where the owner can edit it and everyone else can only read it.
- `755 = rwxr-xr-x`: common for programs and directories that everybody may access, while only the owner may modify them.
- `700 = rwx---`: useful for a private directory or script that only the owner should be able to access or execute.

2. With `chmod 000` you own the file but cannot read it. Why does ownership not help?

Ownership decides which set of permission bits applies to me, but it does not automatically give me access. Since I was the owner, Linux checked the owner bits, and all of them were zero. That is why even though the file belonged to me, `cat` still returned `Permission denied` until I added read permission again.

3. What is your `umask`, and how does it explain the permissions of `new.txt` and `newdir`?

My `umask` was `0002`. A normal new file starts from `666`, and masking out write permission for others gives `664`, which is exactly what I got for `new.txt`: `rw-rw-r-`. A new directory starts from `777`, so with the same mask it became `775`: `rwxrwxr-x`.

4. Explain the `box` experiment: what does `r` allow on a directory, and what does `x` allow?

Read permission on a directory lets me read its list of names, so with only `r` I could run `ls box` and see `f`. However, I could not access `box/f` because I had no execute permission on the directory. With only `x`, the opposite happened: I could not list the directory, but since I

already knew the name `f`, I could traverse the directory and read that file. So for directories, `r` means listing names, while `x` means entering/traversing through the directory.

5. What do the `s` in `/usr/bin/passwd` and the `t` in `/tmp` mean, and why does `passwd` need the `s`?

The `s` on `/usr/bin/passwd` is the setuid bit. It means the program runs with the effective user ID of its owner, which is root here. `passwd` needs this because a normal user must be able to update protected password information even though files such as `/etc/shadow` are not directly readable or writable by that user.

The `t` on `/tmp` is the sticky bit. `/tmp` is writable by many users, but the sticky bit prevents users from deleting or renaming files that belong to somebody else.

5. Part 4 – Disks and the Kernel as Files

Finally, I inspected the physical disk layout and the information Linux exposes through `/proc`. This showed both the real storage underneath the filesystem and how the kernel presents system and process information through file-like interfaces.

Commands

```
lsblk
df -h
mount | grep " / "
grep -c processor /proc/cpuinfo
head -3 /proc/meminfo
head -5 /proc/self/status
ls -l /proc/self/fd
```

5.1. Observations

1. Draw your disk: disk name, partitions, which one is mounted at `/` and with what filesystem type.

My main disk is `sda`, about 40.9 GB in size. It contains two partitions:

Disk layout

```
sda    40.9G disk
|-sda1    1M part
`-sda2 40.9G part /
```

The root filesystem `/` is mounted from `/dev/sda2`, and the filesystem type is `ext4`.

2. How full is your root file system?

The root filesystem was 24% used. The output showed about 8.8 GB used out of a 40 GB filesystem, with around 30 GB still available.

3. How many CPU cores and how much RAM does your laptop have, according

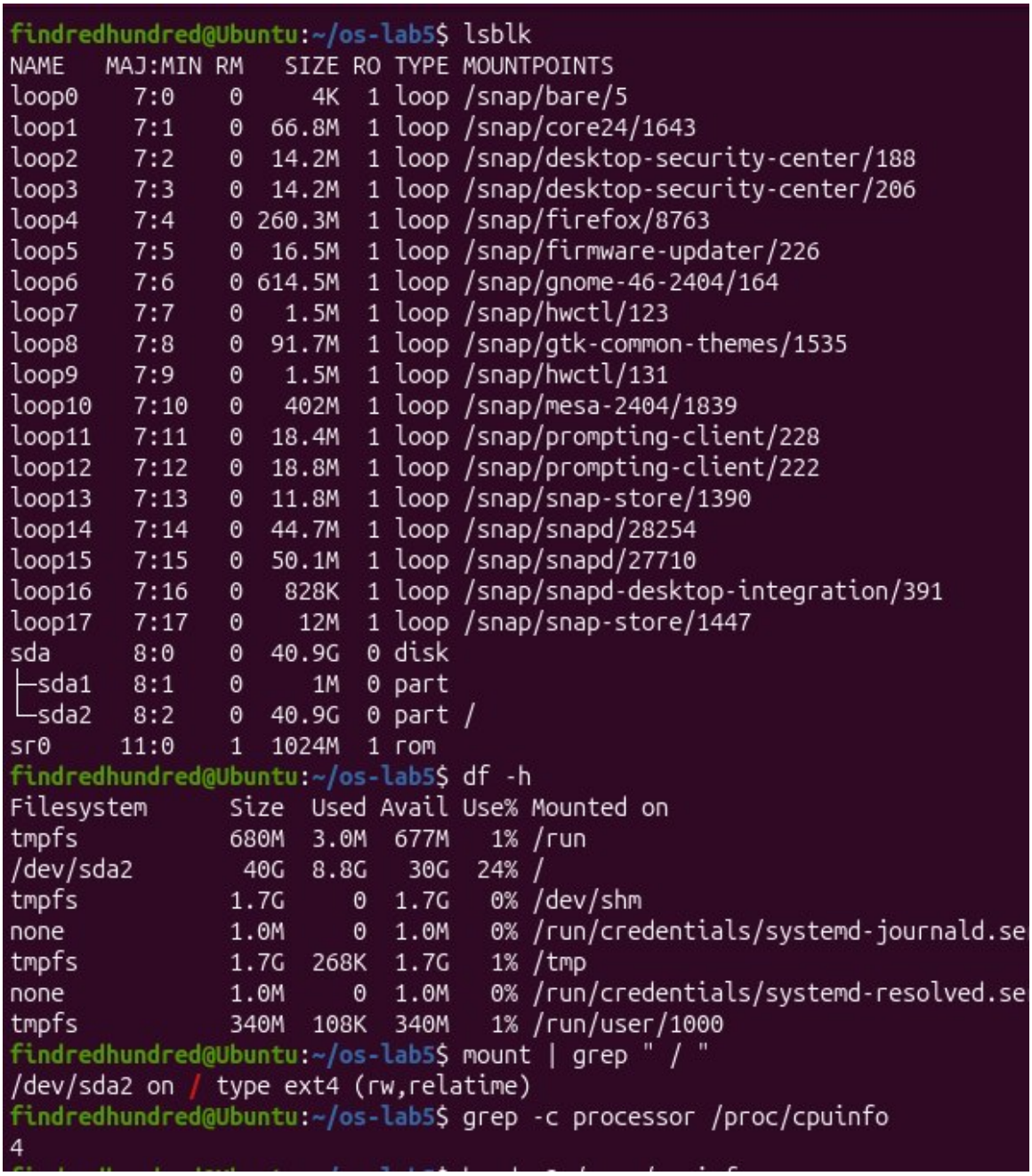


Figure 4.1 – Disk layout, root filesystem usage, mount type, and CPU count.

```
findredhundred@Ubuntu:~/os-lab5$ head -3 /proc/meminfo
MemTotal:        3479664 kB
MemFree:          175804 kB
MemAvailable:     391480 kB
findredhundred@Ubuntu:~/os-lab5$ head -5 /proc/self/status
Name:   head
Umask:  0002
State:  R (running)
Tgid:   74827
Ngid:   0
findredhundred@Ubuntu:~/os-lab5$ ls -l /proc/self/fd
total 0
lrwx------ 1 findredhundred findredhundred 64 Oct 12 19:12 0 -> /dev/pts/0
lrwx------ 1 findredhundred findredhundred 64 Oct 12 19:12 1 -> /dev/pts/0
lrwx------ 1 findredhundred findredhundred 64 Oct 12 19:12 2 -> /dev/pts/0
lr-x------ 1 findredhundred findredhundred 64 Oct 12 19:12 3 -> /proc/74872/fd
findredhundred@Ubuntu:~/os-lab5$
```

Figure 4.2 – Memory information and process information exposed through `/proc`.

to `/proc`?

`/proc/cpuinfo` showed 4 processors, so the system exposes 4 CPU cores/logical processors. `/proc/meminfo` reported `MemTotal:  3479664 kB`, which is roughly 3.3 GiB of RAM.

4. `/proc/self/status` showed the name of a program. Which one, and why that one?

It showed `Name:  head`. This happens because `/proc/self` always refers to the process that is currently accessing it. In the command `head -5 /proc/self/status`, the program opening that file is `head`, so from its point of view `self` means the `head` process itself.

The same idea can be seen with `/proc/self/fd`: it shows the open file descriptors of the process accessing the directory, including standard input, output, and error as descriptors 0, 1, and 2.

6. Conclusion

This lab made the Linux filesystem much easier to understand because every concept was visible directly in the terminal. I saw that file names and inodes are separate, that hard links can keep the same file alive after one name is deleted, while symbolic links only store a path and can become broken. The permission experiments also showed that ownership alone is not enough: Linux still checks the actual permission bits, and directory permissions behave differently from normal file permissions.

The disk and `/proc` part connected the filesystem to the actual hardware and kernel. Overall, the lab showed that Linux exposes a lot of its internal state through the same file-like interface, which makes the whole system feel much more consistent once the ideas of inodes, permissions, mounts, and special files are understood together.